Assignment Day 1
DSA

Name : **Abhinay Kumar**
PRN : **260240120005**
Form No : 251201864
Course : PGCP - AC
Batch : February 2026

Q1. Implement following function to delete element from specified position in given array.
The function should return the updated logical size of array.
int deleteFromPosition(int[] arr, int n, int pos);

```
import java.util.Arrays;

public class Q1_DeleteElement {

   // Function to delete element from specified position in given array
   public static int deleteFromPosition(int[] arr, int n, int pos) {
       if (pos < 0 || pos >= n) {
           System.out.println("Invalid position");
           return n;
       }
       for (int i = pos; i < n - 1; i++) {
           arr[i] = arr[i + 1];
       }
       return n - 1;
   }

   public static void main(String[] args) {
       int[] arr = {10, 20, 30, 40, 50, 0, 0}; // Output array with extra capacity
       int logicalSize = 5;

       System.out.println("Original array: " + Arrays.toString(Arrays.copyOf(arr,
logicalSize)));
       System.out.println("Original logical size: " + logicalSize);

       int deletePos = 2; // Delete element '30'
       logicalSize = deleteFromPosition(arr, logicalSize, deletePos);

       System.out.println("\nAfter deleting element at position " + deletePos +
":");
```

```
        System.out.println("Updated array: " + Arrays.toString(Arrays.copyOf(arr,
logicalSize)));
        System.out.println("Updated logical size: " + logicalSize);
    }
}
```

OUTPUT :

```
(base) abhinyaay@Abhinays-MacBook-Air java-dsa % java Q1_DeleteElement

Original array: [10, 20, 30, 40, 50]
Original logical size: 5

After deleting element at position 2:
Updated array: [10, 20, 40, 50]
Updated logical size: 4
(base) abhinyaay@Abhinays-MacBook-Air java-dsa %
```

Q2. Implement a Stack class using resizable array that implements the Stack interface defined in class.
Throw appropriate exception when required.

Stack.java

```
// Removed package declaration for easier running

public interface Stack {
    void push(int element);
    int pop();
    boolean isEmpty();
    boolean isFull();
}
```

Q2_ResizableStack.java

```
public class Q2_ResizableStack implements Stack {
    private int[] stackData;
    private int top;

    public Q2_ResizableStack(int initialCapacity) {
        stackData = new int[initialCapacity];
```

```java
        top = -1;
    }

    @Override
    public void push(int element) {
        if (isFull()) {
            System.out.println("Stack is full, resizing...");
            resize();
        }
        top = top + 1;
        stackData[top] = element;
    }

    @Override
    public int pop() {
        if (isEmpty()) {
            throw new RuntimeException("Stack Underflow: Stack is empty");
        }
        int result = stackData[top];
        top = top - 1;
        return result;
    }

    @Override
    public boolean isEmpty() {
        return top == -1;
    }

    @Override
    public boolean isFull() {
        return top == (stackData.length - 1);
    }

    private void resize() {
        int newCapacity = stackData.length * 2;
        if (newCapacity == 0) {
            newCapacity = 1;
        }
        int[] newArray = new int[newCapacity];

        for (int i = 0; i <= top; i++) {

        }
        stackData = newArray;
    }
```

```java
    public static void main(String[] args) {
        System.out.println("Testing Resizable Stack:");
        Q2_ResizableStack stack = new Q2_ResizableStack(2);

        stack.push(10);

        System.out.println("Pushed 10, 20. Is the stack full? " + stack.isFull());

        stack.push(30); // Should trigger resize


        System.out.println("Popped: " + stack.pop());

        System.out.println("Popped: " + stack.pop());

        try {

        } catch (RuntimeException e) {
            System.out.println("Exception caught correctly: " + e.getMessage());
        }
    }
}
```

OUTPUT :

```
(base) abhinyaay@Abhinays-MacBook-Air java-dsa % java Q2_ResizableStack

Testing Resizable Stack:
Pushed 10, 20. Is the stack full? false
Popped: 30
Popped: 10
(base) abhinyaay@Abhinays-MacBook-Air java-dsa %
```

Q3. Define Queue as ADT (Refer to class notes).

```java
public interface Q3_QueueADT<T> {
    void enqueue(T element);
    T dequeue();
    T peek();
    boolean isEmpty();
    boolean isFull();
    int size();
```

```
}
```

Q4. Implement a Generic Queue class using array that implements the Queue interface defined in Q1.
Throw appropriate exception when required.
(Refer to class notes)

```java
public class Q4_GenericQueue<T> implements Q3_QueueADT<T> {
    private Object[] queueData;
    private int front;
    private int rear;
    private int size;
    private int capacity;

    public Q4_GenericQueue(int capacity) {
        this.capacity = capacity;
        this.queueData = new Object[capacity];
        this.front = 0;
        this.rear = -1;
        this.size = 0;
    }

    @Override
    public void enqueue(T element) {
        if (isFull()) {
            throw new RuntimeException("Queue Overflow: The queue is full");
        }
        rear = (rear + 1) % capacity;
        queueData[rear] = element;
        size++;
    }

    @Override
    @SuppressWarnings("unchecked")
    public T dequeue() {
        if (isEmpty()) {
            throw new RuntimeException("Queue Underflow: The queue is empty");
        }
        T element = (T) queueData[front];
        queueData[front] = null; // facilitate garbage collection
        front = (front + 1) % capacity;
        size--;
        return element;
```

```java
    }

    @Override
    @SuppressWarnings("unchecked")
    public T peek() {
        if (isEmpty()) {
            throw new RuntimeException("Queue Underflow: The queue is empty");
        }
        return (T) queueData[front];
    }

    @Override
    public boolean isEmpty() {
        return size == 0;
    }

    @Override
    public boolean isFull() {
        return size == capacity;
    }

    @Override
    public int size() {
        return size;
    }

    public static void main(String[] args) {
        System.out.println("Testing Generic Queue (Capacity: 3)");
        Q4_GenericQueue<String> queue = new Q4_GenericQueue<>(3);

        queue.enqueue("First");
        queue.enqueue("Second");
        queue.enqueue("Third");

        System.out.println("Queue size: " + queue.size());

        try {
            queue.enqueue("Fourth"); // Should throw exception
        } catch (RuntimeException e) {
            System.out.println("Exception caught correctly: " + e.getMessage());
        }

        System.out.println("Dequeued: " + queue.dequeue());
        System.out.println("Dequeued: " + queue.dequeue());
```

```java
        queue.enqueue("Fourth"); // Circular wrap-around
        System.out.println("Dequeued: " + queue.dequeue());
        System.out.println("Dequeued: " + queue.dequeue());

        try {
            queue.dequeue(); // Should throw exception
        } catch (RuntimeException e) {
            System.out.println("Exception caught correctly: " + e.getMessage());
        }
    }
}
```

OUTPUT :

```
(base) abhinyaay@Abhinays-MacBook-Air java-dsa % java Q4_GenericQueue

Testing Generic Queue (Capacity: 3)
Queue size: 3
Exception caught correctly: Queue Overflow: The queue is full
Dequeued: First
Dequeued: Second
Dequeued: Third
Dequeued: Fourth
Exception caught correctly: Queue Underflow: The queue is empty
(base) abhinyaay@Abhinays-MacBook-Air java-dsa % java Q4_GenericQueue

Testing Generic Queue (Capacity: 3)
Queue size: 3
Exception caught correctly: Queue Overflow: The queue is full
Dequeued: First
Dequeued: Second
Dequeued: Third
Dequeued: Fourth
Exception caught correctly: Queue Underflow: The queue is empty
(base) abhinyaay@Abhinays-MacBook-Air java-dsa %
```